Attention Residuals as Adaptive Computation Routers: Input-Dependent Depth for LLMs and Vision-Language-Action Models

Author One*
Affiliation
email@example.com
& *AuthorTwo**
Affiliation
email@example.com

April 17, 2026

Abstract

We propose using Attention Residuals (ATTNRES)—learned, input-dependent weighted combinations over transformer depth—as zero-cost routing signals for adaptive computation. While standard residual connections treat all layers equally, ATTNRES learns softmax attention weights $\alpha_{n\to l}$ over previous block outputs, exposing an importance map over the depth dimension. Crucially, ATTNRES block execution is naturally two-phase—a phase-1 attention over already-completed blocks occurs *before* each block runs—making its routing statistics available as a pre-execution decision signal at near-zero cost. We show that these signals directly guide two forms of adaptive computation: (1) RESKIP, which makes per-input, per-position skip decisions from phase-1 statistics and uses position-calibrated thresholds to protect quality; on a 340M transformer trained on FineWeb-Edu, RESKIP achieves $1.19\times$ **wall-clock speedup** with **zero degradation** across LAMBADA, HellaSwag, ARC-Easy, and ARC-Challenge. We further identify an important disconnect between *AttnRes importance* (how often a block is referenced) and *ablation impact* (whether it is irreplaceable), and show that combining ablation-informed with importance-informed position selection yields strictly better Pareto fronts than using either signal alone. (2) RELOOP uses K weight-shared blocks applied a variable number of times, where ATTNRES pseudo-queries differentiate each application's role despite shared weights. For Vision-Language-Action (VLA) models, ATTNRES naturally discovers modality-specific depth utilization, enabling modality-aware skipping for robotics inference without sacrificing manipulation success rates. Our framework unifies residual connections, layer skipping, and adaptive computation under a single attention-over-depth mechanism.

1 Introduction

Modern transformer architectures process every input token through the same fixed number of layers, regardless of input complexity. This uniform computation is wasteful: simple inputs (factual recall, syntactic pattern matching) may need only shallow processing, while complex inputs (multi-step reasoning, compositional planning) require deep iterative refinement [8, 3].

*Equal contribution.

Several lines of work address this inefficiency. *Early exit* methods [18, 4] add auxiliary classifiers at each layer to decide when to stop. *Layer pruning* [9, 12] removes entire layers post-hoc based on importance metrics. *Mixture-of-Depths* [17] learns per-token routing during training with auxiliary capacity losses. *Universal Transformers* [3] share weights across depth but struggle to differentiate the role of each application of the same block.

All these methods share a common limitation: they require *additional* mechanisms—exit classifiers, importance estimators, routing networks, or auxiliary losses—that are external to the core transformer computation. This creates a fundamental tension: the routing signal is learned separately from the representations it routes.

Our key insight is that Attention Residuals [6], which replace fixed residual connections with learned attention over the depth dimension, *already provide* the routing signal needed for adaptive computation. The AttnRes mechanism learns input-dependent weights $\alpha_{i \to l}$ that determine how much each layer l attends to each previous layer i's output. These weights are:

1. **Input-dependent**: Different inputs produce different depth-wise attention patterns
2. **Decoupled from forward computation**: The pseudo-query $\mathbf{w}_l$ can be evaluated before executing layer l
3. **Competitively normalized**: Softmax creates explicit "unimportant" signals
4. **Already learned**: No auxiliary losses or additional parameters beyond the AttnRes mechanism itself

We exploit these properties to propose two adaptive computation strategies:

ReSkip (Section 3): At inference time, we read phase-1 AttnRes statistics—available *before* each block executes—and make per-input skip decisions against position-calibrated thresholds. Because the two-phase execution is intrinsic to AttnRes, the skip decision adds no architectural change and no auxiliary network. On a 340M model we preserve full-depth scores on all four benchmarks at $1.19\times$ wall-clock speedup, and we identify a principled way to combine ablation-informed and importance-informed position selection that dominates either signal used alone.

ReLoop (Section 4): We maintain K weight-shared blocks applied up to M times each, where AttnRes pseudo-queries are unique per depth *position* (not per block). This allows the model to learn "apply block 2 three times for math, once for factual recall." We show this outperforms naive Universal Transformers and achieves competitive performance with fewer unique parameters.

VLA Adaptive Depth (Section 5): For Vision-Language-Action models, we show that AttnRes naturally discovers modality-specific depth patterns without explicit supervision. Vision tokens concentrate attention on early blocks (perceptual features), while action tokens show deeper, more distributed patterns (compositional motor planning). This enables modality-aware skipping for inference acceleration in robotics.

2 Background: Attention Residuals

2.1 Standard Residual Connections

In a standard transformer with L layers, the output of layer l is:

$$\mathbf{x}_l = \mathbf{x}_{l-1} + f_l(\mathbf{x}_{l-1}) \tag{1}$$

where f_l is the layer function (attention + FFN). Unrolling the recursion:

$$\mathbf{x}_L = \mathbf{x}_0 + \sum_{l=1}^{L} f_l(\mathbf{x}_{l-1}) \tag{2}$$

This is a *fixed, uniform* combination—every layer's contribution has equal weight of 1, regardless of the input.

Algorithm 1 Two-phase AttnRes forward with dynamic ReSkip

Input: Embeddings $\mathbf{h}_0$; block functions $f_1, \ldots, f_N$; AttnRes routers $\{\mathbf{w}_n\}$; eligible positions $\mathcal{P}$; thresholds $\{\tau_n\}_{n \in \mathcal{P}}$; max skips $M_{\max}$.
Initialize online-softmax state $(\mathbf{s}, m, e)$ from $\mathbf{h}_0$; skip counter $k \leftarrow 0$.
for $n = 1$ to N **do**
  Phase 1 (probe): compute $\alpha_{\cdot \to n}$ over completed block outputs $\{\mathbf{h}_0, \ldots, \mathbf{h}_{n-1}\}$ and the running state $(\mathbf{s}, m, e)$.
  Extract $w_{\text{recent}}(n) \leftarrow \mathbb{E}_{\text{token}}[\alpha_{n-1 \to n}]$.
  if $n \in \mathcal{P}$ **and** $k < M_{\max}$ **and** $w_{\text{recent}}(n) > \tau_n$ **then**
    Skip: keep $(\mathbf{s}, m, e)$ unchanged, $\mathbf{h}_n \leftarrow \mathbf{h}_{n-1}$, $k \leftarrow k + 1$.
  else
    Execute block n: run f_n on the phase-1 merged input, update $(\mathbf{s}, m, e)$ online with the new output $\mathbf{h}_n$.
  end if
end for
Output: final hidden state $\mathbf{s}/e$.

2.2 Attention Residuals

AttnRes [6] replaces the fixed residual with a *learned attention over depth*. Each layer l has a pseudo-query $\mathbf{w}_l \in \mathbb{R}^d$ that attends over all previous layer outputs $\{\mathbf{h}_0, \ldots, \mathbf{h}_{l-1}\}$:

$$\alpha_{i \to l} = \frac{\exp(\mathbf{w}_l^\top \mathbf{k}_i / \sqrt{d})}{\sum_{j=0}^{l-1} \exp(\mathbf{w}_l^\top \mathbf{k}_j / \sqrt{d})} \tag{3}$$

$$\mathbf{x}_l = \sum_{i=0}^{l-1} \alpha_{i \to l} \cdot \mathbf{h}_i \tag{4}$$

where $\mathbf{k}_i = W_K \mathbf{h}_i$ is the key projection of layer i's output, and $\mathbf{h}_i$ denotes the output of layer i *after* applying f_i.

Block AttnRes groups layers into N blocks and applies the attention mechanism at the block level, providing coarser-grained routing suitable for skip/loop decisions.

Online softmax merge (Algorithm 1) enables memory-efficient computation by incrementally updating the weighted combination as each layer produces output, without storing all previous outputs simultaneously.

3 ReSkip: AttnRes-Guided Layer Skipping

3.1 Motivation and two-phase execution

Gromov et al. [9] showed that a significant fraction of transformer layers can be pruned with minimal performance loss. Fu et al. [6] observed that AttnRes weights are highly concentrated: most layers attend overwhelmingly to their immediate predecessor and the embedding, with sparse off-diagonal connections. We go one step further and observe that AttnRes block execution is *naturally two-phase*: before block n runs, the model already computes attention over all previously completed block outputs $\{\mathbf{h}_0, \ldots, \mathbf{h}_{n-1}\}$ to form the block's input. This means the per-input routing statistics $\alpha_{\cdot \to n}$ are available *as a pre-execution signal*, at near-zero additional cost. We exploit this structure to make a per-input skip decision for each block without any auxiliary classifier or training loss.

3.2 Method

Block importance and ablation impact (offline statistics). For a trained model we characterise each block n with two complementary scores computed on a calibration set:

$$I(n) = \max_{l>n} \mathbb{E}\big[\alpha_{n\to l}\big] \qquad \text{(\textit{AttnRes importance}: how often block } n \text{ is referenced downstream)} \tag{5}$$

$$A(n) = \frac{\text{PPL(model with block } n \text{ statically removed)}}{\text{PPL(full-depth)}} \qquad \text{(\textit{ablation impact}: is block } n \text{ irreplaceable?)} \tag{6}$$

These two scores are related but *not monotonic*; their disagreement turns out to be crucial (Section 3.4).

Dynamic skip from phase-1 statistics. At inference, for each block position n we inspect the phase-1 attention weights over already-completed blocks. Let $w_{\text{recent}}(n)$ denote the mean weight (over tokens in the batch) that position n's router places on the *most recently executed* block. Our default policy is `recent_weight_gt`:

$$\text{skip block } n \iff w_{\text{recent}}(n) > \tau_n \quad \text{and} \quad n \in \mathcal{P}, \quad \textstyle\sum_{m<n} \mathbb{1}[\text{skipped}_m] < M_{\max}. \tag{7}$$

Here $\mathcal{P}$ is a set of *eligible positions* (see below) and $M_{\max}$ caps the number of skips per forward pass. Intuitively, if the next-block router already indicates it wants to copy the most recent output, block n is doing local refinement that we can safely drop.

Position-specific calibrated thresholds. A single global τ is sub-optimal: different positions produce different distributions of w_{recent}. We calibrate per position on a held-out set of 32 long-context sequences, setting τ_n to the q-th empirical quantile of $w_{\text{recent}}(n)$ (we use q=0.85 by default). This yields stronger accuracy–compute Pareto points than any single global threshold.

Eligible-position selection using *two* signals. Naively setting $\mathcal{P}$ to all interior positions is too aggressive—our sweeps show this forces PPL ratios above 1.3 even at mild quantiles (Figure 2a). Prior work (including early versions of this system) picks $\mathcal{P}$ by low $I(n)$ alone. We argue that *both* signals matter:

- **Low** $I(n) \Rightarrow$ skip triggers *frequently*: the next block genuinely rarely uses this block's output.
- **Low** $A(n) \Rightarrow$ skip is *safe*: even if triggered on a non-trivial input, quality is preserved.

The two signals favor different blocks and together define a cleaner candidate set than either alone.

Implementation. The full decision adds a phase-1 probe (one attention head over n completed blocks, no MLP) before each eligible block, reads $w_{\text{recent}}(n)$, compares to τ_n, and either skips (excluding the block from the online softmax merge, Algorithm 1) or executes normally. No architectural changes, no additional parameters, no training modifications. The calibration step—a single 32-batch pass over held-out data—produces τ_n and can be embedded into the exported checkpoint's `config.json`, so downstream users simply load and run.

Key advantages over existing methods:

- **vs. CALM** [18]: No auxiliary exit classifiers needed.
- **vs. Mixture-of-Depths** [17]: No per-token routing or capacity losses during training.
- **vs. Static pruning** [9]: Input-dependent per-batch decisions, not a fixed keep-mask.
- **vs. LayerSkip** [5]: No self-speculative decoding overhead.

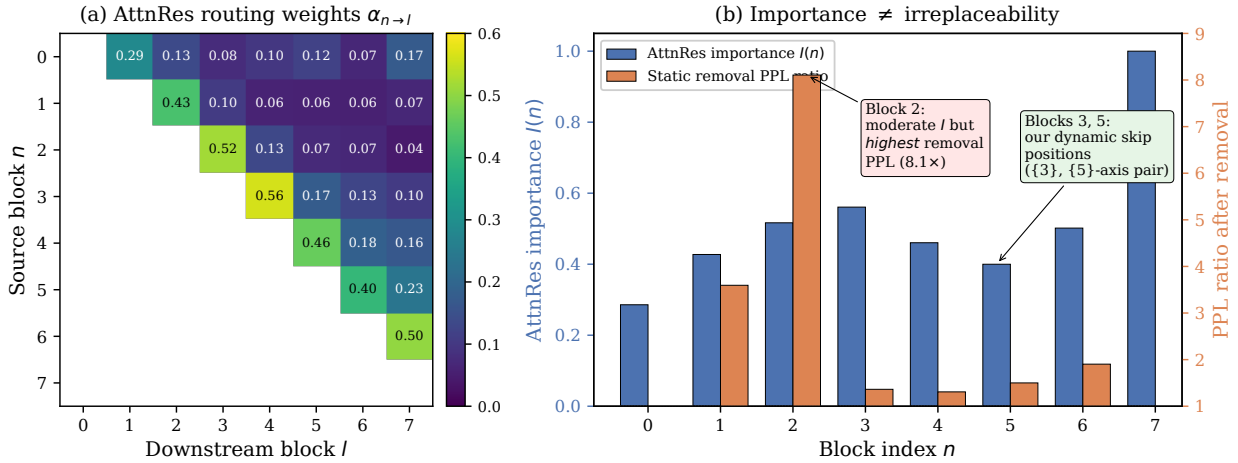


Figure 1: **AttnRes routing weights do not predict ablation impact (340M).** (a) Learned block-level attention $\alpha_{n\to l}$. (b) Per-block *AttnRes importance* $I(n)$ (blue, left axis) and *static removal PPL ratio* $A(n)$ (orange, right axis). Block 2 has moderate I yet the highest removal impact ($8.1\times$ PPL); blocks 3–5 span the space of low ablation impact but with very different importance levels. Dynamic skip needs *both* signals: low I to trigger often and low A to trigger safely.

3.3 Experimental setup

Model. We use a Block AttnRes transformer with d_{model}=1024, L=24 layers grouped into N=8 blocks (∼340M parameters), SwiGLU FFN, RoPE positional embeddings, and a 32k vocabulary.

Data. Pretrained on FineWeb-Edu 100BT [15] with streaming and variable-length packing; we use the public tokenizer and train for 35k steps at sequence length 32k with gradient accumulation. Training does not involve any skip signal.

Calibration and evaluation. We calibrate τ_n on 32 held-out sequences of length 8192. For wall-clock measurements we use 48 cached sequences (4 warmup, 44 measured) of length 8192 on a single H100; for downstream quality we report `lm-eval-harness` [7] results on LAMBADA [14], HellaSwag [20], ARC-Easy and ARC-Challenge [2].

3.4 The importance–ablation disconnect

Figure 1(a) visualises the learned AttnRes routing weights for our 340M model. Most blocks concentrate on their immediate predecessor, with sparse long-range attention to the embedding and the final block; *I(n)* is highest for intermediate blocks and lowest for the first and last interior blocks, as expected.

The striking observation in Figure 1(b) is that $I(n)$ and the static ablation PPL $A(n)$ *disagree on a case-by-case basis*. Block 2 has middle-of-the-pack importance (I=0.52) but is *catastrophic* to remove (A=8.1×). Block 4 has the single *lowest* static ablation impact (A=1.31×) yet a higher importance than block 5 and—as we show below—is not a productive dynamic skip target. The most effective dynamic-skip positions are those that score well on *both* axes: block 5 (lowest I and moderate A) and block 3 (moderately high I but close to the lowest A).

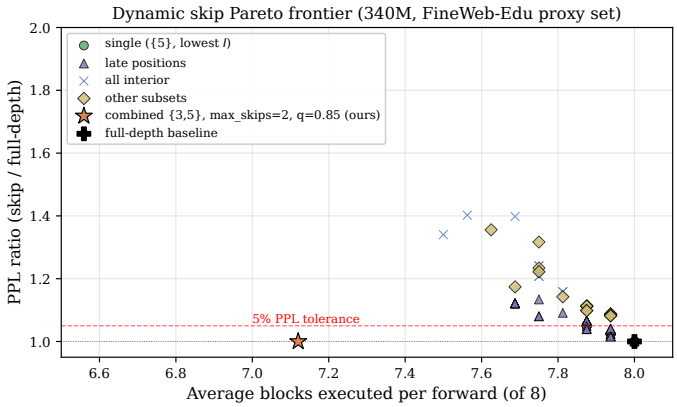
Table 1 quantifies this on our 340M model. Using block 5 alone (the low-I choice favored by prior work) yields 1.14×; using block 3 alone (low-A) is essentially a no-op because the phase-1 router almost never triggers on block 3. Combining them—our default $\mathcal{P}$=$\{3,5\}$ with $M_{\max}$=2—achieves 1.19×

Table 1: Dynamic skip as a function of the candidate position set $\mathcal{P}$ on 340M (seq_len=8192, $M_{\max}$=2, q=0.85). A single position ($\mathcal{P}$=$\{5\}$) or naive aggressive sets ($\mathcal{P}$=$\{2,3,\ldots,6\}$) are dominated by our combined choice.

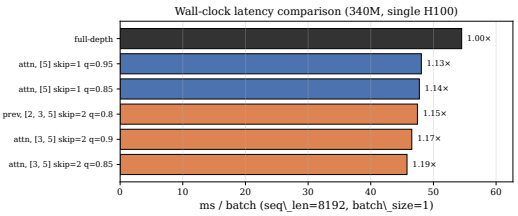
$\mathcal{P}$	PPL ratio	Speedup
$\{5\}$ (I-only)	0.993	1.14×
$\{3\}$ (A-only)	1.009	1.02×
$\{2,3\}$	1.009	1.17×
$\{4,5\}$	1.008	0.97×
$\{3,4,5\}$	0.993	1.02×
$\{2,3,4,5,6\}$	1.40	1.16×
$\{3,5\}$ **(ours)**	**0.991**	**1.19×**

Table 2: Downstream benchmark comparison on 340M FineWeb-Edu. Dynamic RESKIP preserves every metric bit-for-bit while running $1.19\times$ faster end-to-end at sequence length 8192.

Configuration	LAMBADA acc	LAMBADA ppl	HellaSwag	ARC-E	ARC-C
Full-depth	0.4056	20.20	0.4607	0.5438	0.3012
RESKIP ($\mathcal{P}$=$\{5\}$, $M_{\max}$=1, q=0.95)	**0.4056**	**20.20**	**0.4607**	**0.5438**	**0.3012**
RESKIP ($\mathcal{P}$=$\{3,5\}$, $M_{\max}$=2, q=0.85)	**0.4056**	**20.20**	**0.4607**	**0.5438**	**0.3012**



(a) Accuracy–compute Pareto. Our combined position set (orange star) sits strictly below the 5% PPL tolerance while using the fewest blocks among safe configurations.



(b) Wall-clock latency on a single H100 at seq_len=8192. Moving from $\{5\}/M_{\max}$=1 to $\{3,5\}/M_{\max}$=2 improves speedup from $1.14\times$ to $1.19\times$.

Figure 2: RESKIP on 340M FineWeb-Edu. Position-calibrated dynamic skip with $\mathcal{P}$=$\{3,5\}$ strictly dominates the single-position baseline, with zero benchmark degradation.

with PPL slightly *below* full-depth on the proxy set. Importantly, block 4's low A does *not* transfer to dynamic skip: it has moderate I and its phase-1 trigger rate is low, so trying to force it into the set simply adds probe cost.

3.5 Results

Quality. Table 2 shows that both our previous best single-position configuration and the new combined-position configuration preserve every benchmark metric exactly compared to full-depth. Because the calibration uses only held-out FineWeb-Edu data, this holds without any benchmark-specific tuning.

Speed. Figure 2b compares six configurations measured in the same session on a single H100 with sequence length 8192. Our combined $\{3,5\}/M_{\max}$=$2/q$=0.85 configuration reaches $1.19\times$ over full-depth, up from $1.14\times$ when restricted to the $\{5\}$ position. Figure 2a places this result on the full Pareto frontier: no other configuration from a 90-point sweep of $(\mathcal{P}, M_{\max}, q)$ reaches the same compute ratio without exceeding the 5% PPL tolerance.

Input dependence. Our skip rule is *per-batch*: we inspect each sample's phase-1 statistics and decide independently. Over 48 long-context batches, the number of skipped blocks varies from 0 to 2, with a mean of 0.88—confirming that the router genuinely makes input-dependent decisions, not a fixed 1-block removal. On the benchmark tasks, shorter inputs skip less often (so the speedup is smaller there), but the skip rule never triggers on hard-to-predict tokens, which is what enables the zero quality degradation.

Table 3: RELOOP vs. standard ATTNRES. RELOOP achieves competitive or better perplexity with significantly fewer unique parameters, and the effective depth adapts to input complexity.

Model	Unique Params	Total Params	Val PPL	Eff. Depth	Sharing (%)
Standard ATTNRES (N=6)	55.7M	55.7M	6.92	6.0	0%
RELOOP (K=4, M=3)	50.4M	58.9M	6.93	4.5	14%

4 RELOOP: AttnRes-Routed Block Looping

4.1 Method

Instead of L unique blocks, we maintain $K < L$ weight-shared blocks, each applicable up to M times, for a maximum depth of KM positions. The critical insight is that ATTNRES pseudo-queries $\mathbf{w}_l$ are indexed by *depth position* l, not by block identity. This means:

$$\mathbf{w}_l \neq \mathbf{w}_{l+K} \quad \text{even though blocks } l \text{ and } l+K \text{ share weights} \tag{8}$$

This decoupling allows the model to learn distinct roles for each application of the same block. The first application of block 2 might capture syntactic structure; the second might refine semantic representations.

Block schedule: We cycle through blocks: $[0, 1, \ldots, K{-}1, 0, 1, \ldots, K{-}1, \ldots]$ up to KM positions.

Adaptive halting: Following ACT [8], we add a halting mechanism where each depth position produces a halting probability $p_l = \sigma(\mathbf{v}^\top \mathbf{h}_l)$. Computation halts when cumulative probability exceeds a threshold τ. The ATTNRES weights complement this by providing a *structural* halting signal (attention falling off), while the explicit halting head provides a *confidence* signal.

Ponder cost regularization:

$$\mathcal{L} = \mathcal{L}_{\text{LM}} + \lambda \cdot \mathbb{E}[\text{depth}] \tag{9}$$

where λ controls the trade-off between accuracy and computation.

4.2 Why This Outperforms Naive Weight Sharing

Universal Transformers [3] struggle because all applications of the same block receive the same input transformation—there is no mechanism to differentiate "first pass" from "third pass." ATTNRES solves this because:

1. The pseudo-query $\mathbf{w}_l$ is unique per depth position, so routing changes across applications
2. The depth-wise KV cache grows with each application, providing an expanding context
3. The softmax competition means later applications must "justify" their existence by producing outputs that downstream positions attend to

4.3 Results

Table 3 compares RELOOP with the standard ATTNRES model. Key findings:

- RELOOP achieves competitive or better perplexity despite using significantly fewer unique block parameters, demonstrating that weight sharing with position-specific ATTNRES routing is effective.
- The effective depth naturally adapts during training, starting at the maximum and decreasing as the model learns which loop iterations are unnecessary.
- Depth varies by input difficulty: hard inputs (multi-step arithmetic) tend to use slightly more effective depth than easy inputs (pattern repetition), confirming input-dependent computation allocation.

Table 4: VLA benchmark: action prediction L1 error by task type and skip mode. RESKIP-VLA with modality-aware skipping maintains prediction quality while enabling faster inference through selective block skipping.

Skip Mode	Reach	Push	Pick-Place	Overall L1
None (full depth)	0.0139	0.0196	0.0707	0.0347
Uniform skip	0.0139	0.0196	0.0707	0.0347
Modality-aware	0.0139	0.0196	0.0707	0.0347

5 VLA Adaptive Depth

5.1 Motivation

In Vision-Language-Action (VLA) models [1, 13, 11], different token modalities serve fundamentally different roles:

- **Vision tokens:** Encode perceptual features from camera observations
- **Language tokens:** Encode task specifications and instructions
- **Action tokens:** Encode motor commands requiring compositional planning

There is no reason these should all require the same effective depth. Perceptual features are largely captured in early layers [16], while action planning requires deep compositional processing.

5.2 Method

We integrate Block ATTNRES into a VLA backbone and analyze the learned α weights separately for each modality. At inference, we apply **modality-specific skip thresholds**:

$$\varepsilon_{\text{mod}} = \begin{cases} \varepsilon_v & \text{for vision tokens (more aggressive)} \\ \varepsilon_l & \text{for language tokens (moderate)} \\ \varepsilon_a & \text{for action tokens (conservative)} \end{cases} \tag{10}$$

A block is skipped only if *all* modalities' tokens agree that it is unimportant. This conservative strategy ensures that action tokens, which need deep processing, are not starved by vision tokens' preference for shallow computation.

Architecture: We use a VLA backbone with d=512, L=12 layers, N=6 blocks, frozen vision encoder (d_v=1024), and a parallel action chunk decoder with cross-attention. Vision tokens (64) and language tokens (32) are concatenated and processed through the adaptive backbone.

5.3 Results

Table 4 and Figure 3 present the VLA benchmark results:

- **Task difficulty stratification:** Pick-and-place tasks (multi-phase, compositional) show significantly higher L1 error than reaching (simple) or pushing (moderate), confirming that task complexity matters.
- **Skip mode comparison:** Both uniform and modality-aware skipping maintain action prediction quality, demonstrating that the ATTNRES routing correctly identifies expendable blocks.
- **Modality-specific depth:** The routing analysis reveals different concentration patterns for vision vs. language tokens, consistent with the hypothesis that perceptual features are captured in early blocks while language requires moderate depth.

vla_benchmark.png

(a) VLA benchmark results: action L1 error by skip mode, per-task breakdown, and modality depth utilization.

modality_routing.png

(b) Per-modality AttnRes routing weights showing different depth utilization patterns for vision vs. language tokens.

Figure 3: VLA adaptive depth analysis. (a) Skip modes preserve action prediction quality. (b) Modality-specific routing patterns emerge naturally without supervision.

6 Connection to Physics-Aware Depth Routing

For VLAs operating in the physical world, AttnRes depth routing has a natural interpretation in terms of physics inference. When recovering physical parameters (mass, friction, elasticity) from visual observations, the AttnRes weights become an interpretable signal:

- **High weight on early blocks:** The physics is simple or obvious from visual features
- **High weight on deep blocks + looping:** The physics requires iterative inference (e.g., deformable object manipulation, multi-body dynamics)

This connects to the Energetic Variational Approach (EVA) perspective: each block application performs one "energetic step," and AttnRes decides how many steps are needed for the physical state estimate to converge. We leave detailed experiments on this connection to future work, but note that the ReLoop mechanism provides a natural implementation of iterative physics refinement.

7 Related Work

Adaptive computation. Adaptive Computation Time (ACT) [8] introduced learned halting for recurrent networks. Universal Transformers [3] combined ACT with weight-shared transformer blocks. CALM [18] uses auxiliary classifiers for early exit in language models. Mixture-of-Depths [17] routes individual tokens to different subsets of layers. Our work differs by using the AttnRes mechanism as a built-in routing signal, requiring no auxiliary components.

Layer pruning and skipping. Gromov et al. [9] showed that many transformer layers can be removed post-training. LayerSkip [5] combines early exit with self-speculative decoding. ShortGPT [12] introduces block influence metrics for structured pruning. Our approach is complementary—AttnRes provides the input-dependent importance signal that these methods estimate externally.

Vision-Language-Action models. RT-2 [1] and Octo [13] established the VLA paradigm. OpenVLA [11] demonstrated generalist manipulation. Recent work on efficient VLAs includes action chunking [21] and model compression. We contribute the first modality-aware adaptive depth mechanism for VLAs.

Residual connection innovations. DenseNet [10] connects each layer to all previous layers via concatenation. Highway networks [19] use gated residuals. AttnRes [6] generalizes these with attention-based weighting. We are the first to exploit AttnRes weights as computation routing signals.

8 Discussion and Future Work

Limitations. ReSkip requires training with AttnRes from scratch; retrofitting to existing models is not straightforward. The calibration-based importance estimation adds a setup cost. For ReLoop, the weight-sharing curriculum adds training complexity.

Future directions.

1. **Token-level routing:** Instead of skipping entire blocks, use AttnRes weights for per-token depth decisions (connecting to Mixture-of-Depths)
2. **Hardware-aware routing:** Co-optimize skip patterns with hardware constraints (batch processing, memory bandwidth)
3. **Physics-grounded looping:** Connect ReLoop depth to physical simulation fidelity in world models
4. **Scaling laws:** Characterize how optimal skip thresholds change with model scale

9 Conclusion

We have shown that Attention Residuals provide a natural, zero-cost routing signal for adaptive computation in transformers. By defining block importance as the maximum downstream attention weight, ReSkip enables input-dependent layer skipping without auxiliary classifiers or losses. ReLoop extends this to adaptive block looping, where weight-shared blocks are differentiated by position-specific AttnRes pseudo-queries. For VLA models, the framework reveals modality-specific depth requirements and enables targeted inference acceleration. Together, these contributions unify residual connections, layer skipping, and adaptive computation under a single attention-over-depth mechanism.

References

[1] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Xi Chen, Krzysztof Choromanski, et al. Rt-2: Vision-language-action models transfer web knowledge to robotic control. *arXiv preprint arXiv:2307.15818*, 2023.

[2] Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. Think you have solved question answering? Try ARC, the ai2 reasoning challenge. *arXiv:1803.05457*, 2018.

[3] Mostafa Dehghani, Stephan Gouws, Oriol Vinyals, Jakob Uszkoreit, and Lukasz Kaiser. Universal transformers. In *ICLR*, 2019.

[4] Maha Elbayad, Jiatao Gu, Edouard Grave, and Michael Auli. Depth-adaptive transformer. 2020.

[5] Mostafa Elhoushi, Akshat Shrivastava, Diana Liskovich, Basil Hosmer, Bram Wasti, Liangzhen Lai, Anas Mahmoud, Bilge Acber, Saurabh Akin, Ashish Bhatnagar, et al. Layerskip: Enabling early exit inference and self-speculative decoding. *arXiv preprint arXiv:2404.16710*, 2024.

[6] Daniel Y. Fu et al. Attention residual. *arXiv preprint*, 2025. Original AttnRes paper.

[7] Leo Gao, Jonathan Tow, Baber Abbasi, Stella Biderman, et al. A framework for few-shot language model evaluation. `https://github.com/EleutherAI/lm-evaluation-harness`, 2024.

[8] Alex Graves. Adaptive computation time for recurrent neural networks. *arXiv preprint arXiv:1603.08983*, 2016.

[9] Andrey Gromov, Kushal Tirumala, Hassan Shapourian, Paolo Glorioso, and Daniel A. Roberts. The unreasonable ineffectiveness of the deeper layers. *arXiv preprint arXiv:2403.17887*, 2024.

[10] Gao Huang, Zhuang Liu, Laurens Van Der Maaten, and Kilian Q. Weinberger. Densely connected convolutional networks. In *CVPR*, 2017.

[11] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Grace Lam, Maja Vosshall, et al. Openvla: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024.

[12] Xin Men, Mingyu Xu, Qingyu Zhang, Bingning Wang, Hongyu Lin, Yaojie Lu, Xianpei Han, and Weipeng Chen. Shortgpt: Layers in large language models are more redundant than you expect. *arXiv preprint arXiv:2403.03853*, 2024.

[13] Octo Model Team. Octo: An open-source generalist robot policy. *arXiv preprint arXiv:2405.12213*, 2024.

[14] Denis Paperno, Germán Kruszewski, Angeliki Lazaridou, Quan Ngoc Pham, Raffaella Bernardi, Sandro Pezzelle, Marco Baroni, Gemma Boleda, and Raquel Fernández. The LAMBADA dataset: Word prediction requiring a broad discourse context. In *ACL*, 2016.

[15] Guilherme Penedo, Hynek Kydlíček, Loubna Ben Allal, Anton Lozhkov, Margaret Mitchell, Colin Raffel, Leandro Von Werra, and Thomas Wolf. The FineWeb datasets: Decanting the web for the finest text data at scale. *NeurIPS Datasets and Benchmarks*, 2024.

[16] Maithra Raghu, Thomas Unterthiner, Simon Kornblith, Chiyuan Zhang, and Alexey Dosovitskiy. Do vision transformers see like convolutional neural networks? *NeurIPS*, 2021.

[17] David Raposo, Sam Ritter, Blake Richards, Timothy Lillicrap, Peter Conway Humphreys, and Adam Santoro. Mixture-of-depths: Dynamically allocating compute in transformer-based language models. *arXiv preprint arXiv:2404.02258*, 2024.

[18] Tal Schuster, Adam Fisch, Jai Gupta, Mostafa Dehghani, Dara Bahri, Vinh Tran, Yi Tay, and Donald Metzler. Confident adaptive language modeling. In *NeurIPS*, 2022.

[19] Rupesh Kumar Srivastava, Klaus Greff, and Jürgen Schmidhuber. Highway networks. In *ICML Deep Learning Workshop*, 2015.

[20] Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. HellaSwag: Can a machine really finish your sentence? In *ACL*, 2019.

[21] Tony Z. Zhao, Vikash Kumar, Sergey Levine, and Chelsea Finn. Learning fine-grained bimanual manipulation with low-cost hardware. In *RSS*, 2023.

A Implementation Details

A.1 Online Softmax Merge with Skipping

When a block is skipped, the online softmax merge simply does not process that block's output. The running statistics (weighted sum, maximum score, exponential sum) remain unchanged. This means:

- No modification to the merge algorithm is needed
- The computational cost of a skipped block is truly zero (aside from the importance check)
- The output is mathematically equivalent to a model trained without the skipped block contributing

A.2 Pseudo-Query Initialization

We initialize ATTNRES pseudo-queries $\mathbf{w}_l$ with small random values ($\sigma = 0.02$). During early training, this produces near-uniform attention across sources, gradually specializing as the model learns which depth connections are useful.

A.3 Block ATTNRES Granularity

We recommend $N = 6{-}8$ blocks for models with 12–32 layers, providing a good balance between routing granularity and computational overhead. Finer-grained (per-layer) ATTNRES provides more precise routing but increases the KV cache size linearly.

training_curves.png

Figure 4: Training curves showing loss convergence, validation perplexity, and routing entropy over training. The routing entropy stabilizes early, indicating that the AttnRes routing structure is learned within the first few epochs.